SWE30011 - IoT Programming

Week 1 Tutorial

Getting Started with Arduino

Objective

The objective of this tutorial is to familiarize yourself with Arduino. Arduino is an open-source electronics platform that consists of multiple components, including a microcontroller, power supply, digital and analog I/O pins, USB interface, and reset button. There is a wide range of Arduino models, such as Arduino Uno, Mega, Nano, Micro, Leonardo, and Due. Each of these models differs in characteristics such as the number of pins, processor speed, memory capacity, operating voltage, and available communication interfaces. In this unit, we will use Arduino UNO, which is shown in Figure 1. In this tutorial, we focus on developing a traffic light system for both vehicles and pedestrians at an intersection. Let's get started.

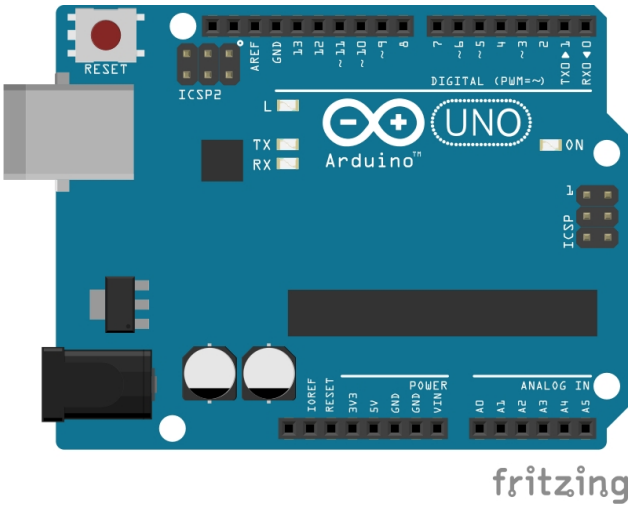


Figure 1: Arduino UNO Board

Task 1: Setting up Arduino

In order to start using Arduino and developing IoT projects, an Integrated Development Environment (IDE) is required. Arduino provides its own IDE, which can be installed using the following link for different operating systems.

- **Arduino IDE**: https://www.arduino.cc/en/software

Once the IDE is installed, you will have access to an environment illustrated in Figure 2.

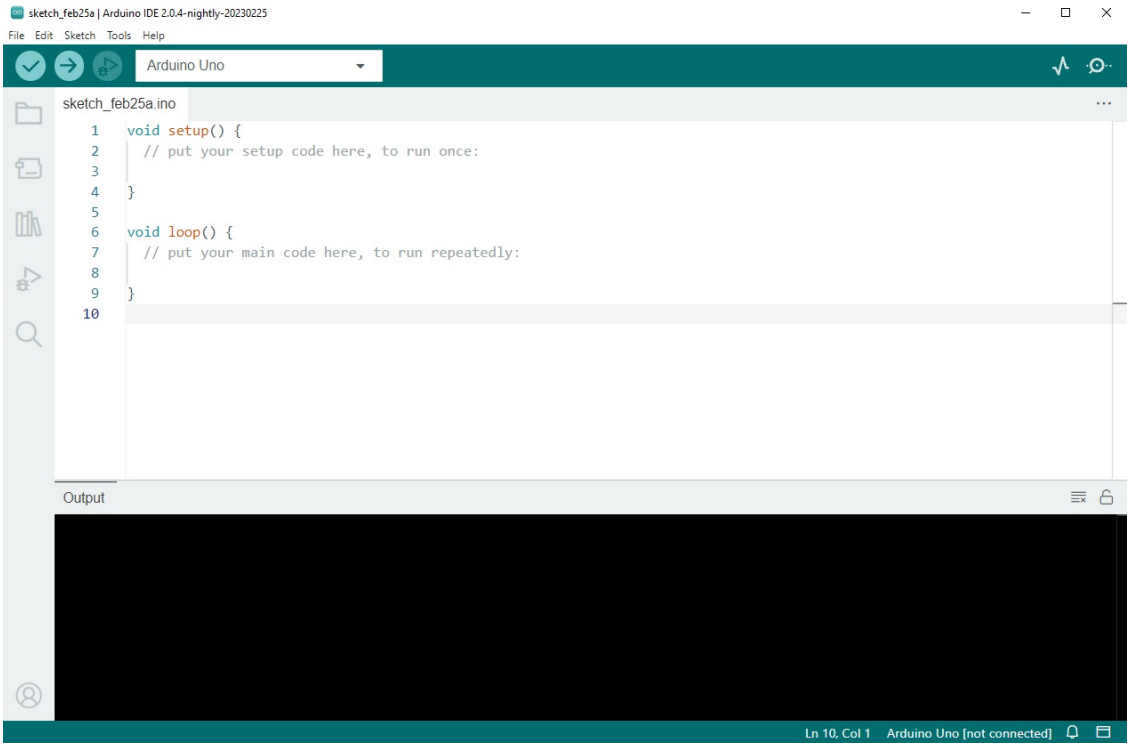


Figure 2: Arduino IDE

The next step is to connect the Arduino board to the IDE. To do so, follow these three steps:

- **Step 1**: Connect the Arduino board to the laptop using a USB cable.
- **Step 2**: Select the board type via the menu
  Tools > *Board* > *Arduino AVR Boards* > *Arduino UNO*.
- **Step 3**: Select the port to which the Arduino board is connected via the menu
  Tools > *Port* > *Connected Port*.

After connecting the Arduino board to the IDE, it's time to start coding. There are two main functions in an Arduino code which are the `setup()` and `loop()` functions as illustrated in Figure 2. The `setup()` function runs once at the beginning of the program and is used to initialize settings such as pin modes, serial communication, and sensor configurations. After `setup()` function executes, the `loop()` function continuously runs, executing the main logic of the program. This function is responsible for handling tasks like reading sensor data, controlling outputs, and responding to user inputs. Together, `setup()` and `loop()` functions structure the Arduino code to ensure proper initialization and continuous operation.

Task 2: Blinking LED

In this task, you are required to set up a circuit with an LED connected to digital PIN 2 and program it to blink at a one-second interval. Figure 3 illustrates the circuit that needs to be assembled before writing the code for the application.

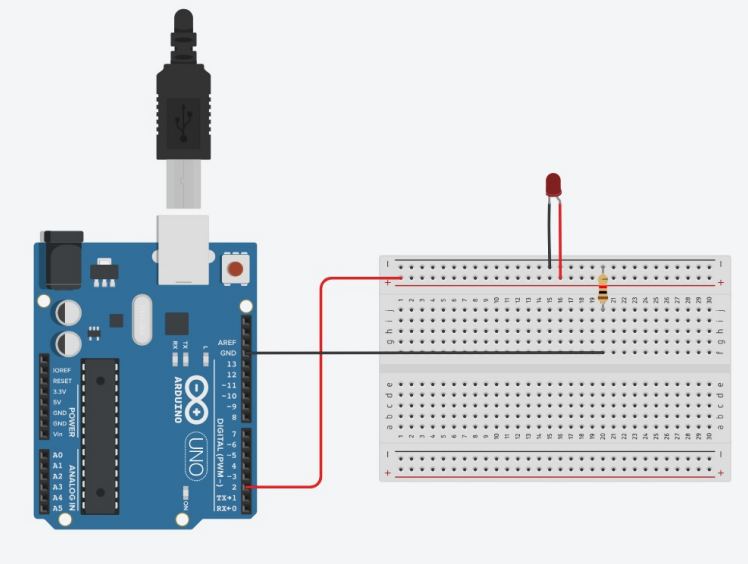


Figure 3: Blinking LED Circuit

Note: An LED has two pins. The longer pin, also known as the positive pin or *anode*, should be connected to the digital pin, while the shorter pin, also known as the negative pin or *cathode*, should be connected to the ground (GND) pin.

Note: There is need to use a resistor in the circuit to limit the current flowing through the LED and prevent damage to both the LED and the Arduino pin.

Once the circuit is assembled properly, it is time to code the application. First, we need to initialize PIN 2, which is connected to the positive pin of the LED, as an output pin. To

do so, we call the `pinMode()` function inside the `setup()` function of the Arduino code and define it as `OUTPUT`:

```
pinMode(2, OUTPUT);
```

Next, we need to send a high signal from PIN 2 to turn on the LED, which can be achieved by calling the `digitalWrite()` function inside the `loop()` function:

```
digitalWrite(2, HIGH);
```

After that, we need to keep the LED on for one second, which can be done using the `delay()` function. The `delay()` function works in milliseconds, so we must include the value `1000` for a one-second delay. Then, we need to turn off the signal from PIN 2 again using the `digitalWrite()` function:

```
digitalWrite(2, LOW);
```

Finally, we keep the LED off for a second using the `delay()` function.

After writing the code, compile it using the tick symbol in the top-left corner of the Arduino IDE, then upload it using the right arrow icon in the top-left corner. Now, the LED should blink.

Task 3: Traffic Light

Now that we know how to make an LED blink, why not create a traffic light? To do so, we need to add two more LEDs to the previous circuit and connect them to different digital pins of our choice. Figure 4 illustrates the circuit of a traffic light.

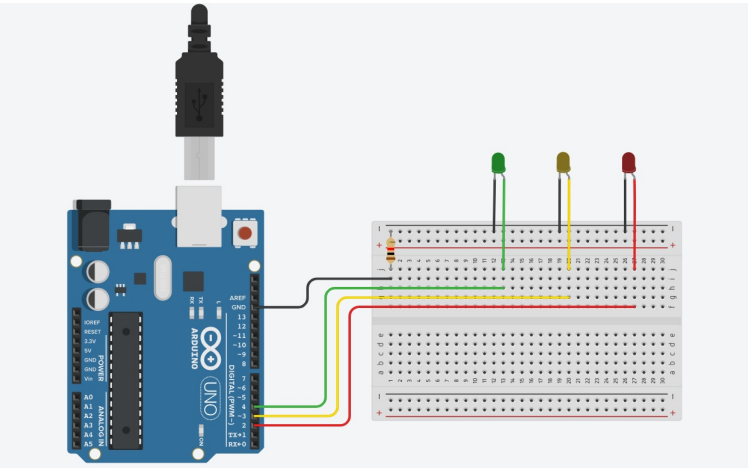


Figure 4: Traffic Light Circuit

After setting up the circuit, use the functions from Task 2 to initialize the traffic light. Let's make the red light stay on for 5 seconds, the green light for 5 seconds, and the yellow light for 2 seconds. Ensure the lights turn on and off in proper order.

Task 4: Traffic Light with Buzzer

Now, let's improve our traffic light by adding a buzzer. We will use the buzzer to inform pedestrians when they can cross the intersection, which is when the traffic light is red for vehicles. Figure 5 shows the circuit of the traffic light with a buzzer.

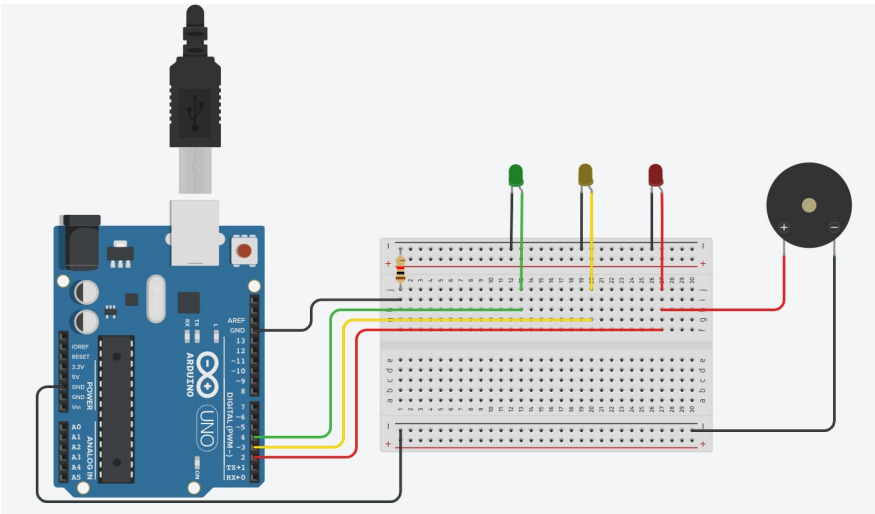


Figure 5: Traffic Light with a Buzzer Circuit

Do we need to make any changes to the code for this task?

Task 5: Fully Functional Traffic Light System for Vehicles and Pedestrians

For the final task, let's improve the traffic light so that it includes lights for both vehicles and pedestrians. To complete our traffic light and make it fully functional for both vehicles and pedestrians, we need to add two more LEDs for the pedestrians. Let's connect a red and green LED to two additional digital pins of our choice, as illustrated in Figure 6.

Now, using the previously used functions, ensure that the green light for pedestrians is on when the red light for vehicles is on, and the red light for pedestrians is on when the green and yellow lights for vehicles are on.

Congratulations! You have developed your first IoT application of the semester.

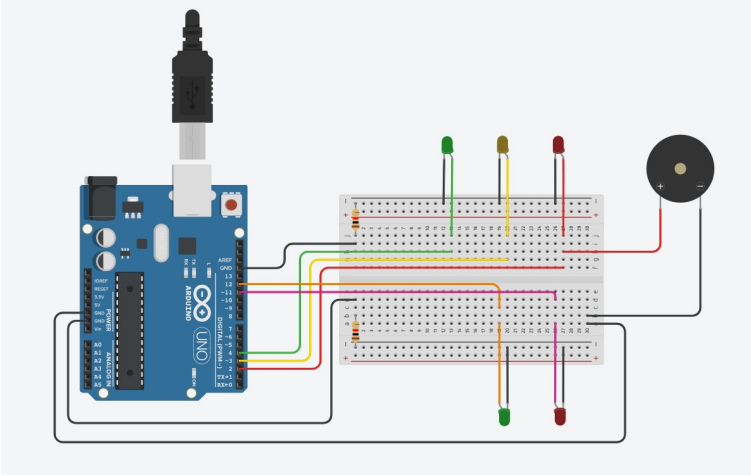


Figure 6: Traffic Light System for Vehicles and Pedestrians